

Seminar Report

Security Analysis of Ephemeral Kubernetes in HPC Environments

Shrinath Madde

MatrNr: 25235677

Supervisor: Jonathan Decker

Georg-August-Universität Göttingen
Institute of Computer Science

February 25, 2026

Abstract

While Ephemeral Kubernetes offers rapid, sub-90-second cluster deployments for High-Performance Computing (HPC) environments, its original architecture inherently prioritizes speed over security. By relying on shared Network File System (NFS) storage for cluster bootstrapping, the system exposes critical cryptographic materials, most notably full cluster-admin credentials, creating severe risks of persistent credential exposure and privilege escalation. This paper analyzes these fundamental security flaws and proposes two architectural enhancements to secure the cluster lifecycle. First, we introduce a Key-Based Credential Distribution system that eliminates persistent storage risks through authenticated, transient file delivery. Second, we implement a Token-Based Access Control model that enforces the principle of least privilege by replacing unrestricted admin credentials with scoped join tokens. Together, these methodologies ensure that Ephemeral Kubernetes can satisfy the rigorous security demands of multi-tenant HPC facilities without sacrificing its signature agility and performance.

Declaration on the use of ChatGPT and comparable tools in the context of examinations

In this work I have used ChatGPT or another AI as follows:

- ☐ Not at all
- ☒ During brainstorming
- ☐ When creating the outline
- ☐ To write individual passages, altogether to the extent of 0% of the entire text
- ☐ For the development of software source texts
- ☒ For optimizing or restructuring software source texts
- ☒ For proofreading or optimizing
- ☐ Further, namely: -

I hereby declare that I have stated all uses completely.

Missing or incorrect information will be considered as an attempt to cheat.

Contents

List of Tables	iv
List of Figures	iv
List of Abbreviations	v
1 Introduction	1
2 Background and Security Challenges	2
2.1 Ephemeral Kubernetes Architecture Overview	2
2.2 Cluster Formation and Node Join Process	2
2.2.1 Leader Election & Cluster Initialization	3
2.2.2 Follower and Worker Node Join Process	3
2.3 Node Rejoin and Recovery Mechanism	3
3 Security Vulnerabilities in the Ephem- eral Model	4
4 PKI Certificates and Their Usage	4
5 Proposed Security Enhancement: Key-Based Credential Distribution	5
5.1 Architecture Overview	5
5.2 Key Generation and Distribution	5
5.3 Secure Credential Upload Mechanism	6
5.4 Key-Authenticated Download and Self-Destruction	6
5.5 Key Lifecycle and Node Rejoin Mechanism	7
5.6 Security Improvements	7
5.7 Implementation Considerations	8
6 Alternative Approach: Token-Based Access Control	8
6.1 Recommended Files for Shared Storage	9
6.1.1 Bootstrap Join Token (<code>join-command.txt</code>)	9
6.1.2 Certificate Key (<code>certificate-key.txt</code>)	9
6.1.3 Phylactery Service Account Token (<code>phylactery-token.txt</code>)	9
6.2 Security Benefits	10
7 Discussion	10
8 Conclusion	11
9 References	11
A Work sharing	A1
A.1 Hans	A1
A.2 Peter	A1
B Code samples	A1

List of Tables

List of Figures

- 1 Unsecured ephemeral Kubernetes setup: The Kube.config (used to join the cluster) is directly accessible through the /share folder; anyone can use it and deploy privileged pods and get root access on the node. Figure created using <https://app.diagrams.net/> 1
- 2 Kube.config will be saved in a secure server folder on the cluster manager, which is accessible using Key(separate for each node, provided through overlay) authentication. Figure created using <https://app.diagrams.net/> . . . 6

List of Abbreviations

API	Application Programming Interface
CA	Certificate Authority
CN	Common Name
GPU	Graphics Processing Unit
HPC	High-Performance Computing
HTTP	Hypertext Transfer Protocol
KSI	Kind Slurm Integration
MPI	Message Passing Interface
NFS	Network File System
PKI	Public Key Infrastructure
SHA	Secure Hash Algorithm
VM	Virtual Machine
WW	Warewulf

1 Introduction

While Kubernetes is not as popular in production HPC environments as it is in cloud VMs due to complexity in adaptability, maintainability, performance bottlenecks, and security, several genuine approaches attempt to fill this gap.

Zhou et al. [5] use Torque-Operator to bridge Kubernetes and HPC clusters managed by TORQUE, enabling container orchestration while keeping the HPC environment intact. However, this requires persistent external components outside Slurm jobs, adding infrastructure complexity and security concerns in multi-user environments. Chazapis et al. [2] run Slurm under Kubernetes, converting user services into Slurm scripts that execute as Singularity containers on HPC compute nodes. This approach does not support Kubernetes ClusterIP services, lacks GPU mapping and port forwarding, and relies on static Slurm allocations, limiting scheduling flexibility. Decker et al. [3] introduce KSI (Kind Slurm Integration), which deploys complete Kubernetes clusters within Slurm jobs, providing full Kubernetes features. However, KSI only supports single-node deployments, lacks multi-node MPI-style parallel jobs, and suffers from poor network performance.

Ephemeral Kubernetes[1] addresses challenges in adaptability and maintainability by integrating container orchestration directly into existing HPC stacks via Warewulf with minimal architectural changes. This framework provides native Kubernetes performance and services, and could also be used to dynamically negotiate HPC cluster capacity between on-demand and batch clusters[4]. The current Ephemeral Kubernetes implementation prioritises deployment speed and operational simplicity over security, relying on shared Network File System (NFS) storage to distribute sensitive cryptographic materials. Specifically, the Kubernetes cluster-administrator configuration file (`kube.config`) containing full cluster-admin credentials is stored on NFS accessible to all cluster nodes. This architectural decision creates severe security vulnerabilities:

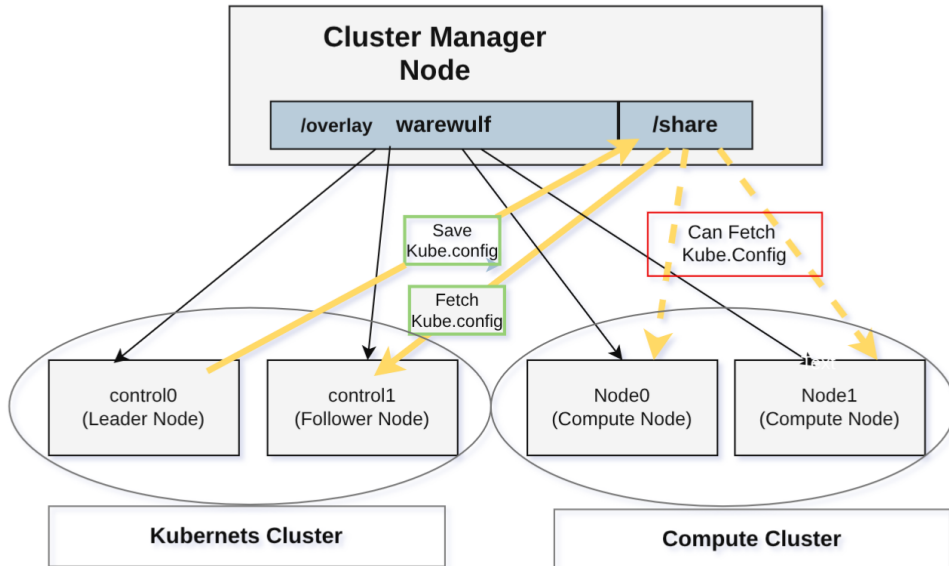


Figure 1: Unsecured ephemeral Kubernetes setup: The `Kube.config` (used to join the cluster) is directly accessible through the `/share` folder; anyone can use it and deploy privileged pods and get root access on the node. Figure created using <https://app.diagrams.net/>

- **Credential Exposure:** Any compromised node or malicious container with NFS access can steal cluster-admin credentials.
- **Privilege Escalation:** Stolen credentials enable deployment of privileged containers, leading to root access on any cluster node.
- **Persistent Risk:** Credentials remain exposed indefinitely on NFS throughout the cluster lifecycle.
- **No Revocation:** Once credentials are copied, they cannot be revoked or expired.
- **Lateral Movement:** Compromising a single worker node provides a path to compromise the entire cluster.

These vulnerabilities are unacceptable for production HPC environments, particularly those serving multiple tenants, processing sensitive data, or subject to compliance Requirements.

2 Background and Security Challenges

2.1 Ephemeral Kubernetes Architecture Overview

Ephemeral Kubernetes is built upon Warewulf (WW), a stateless node provisioning system commonly deployed in HPC environments. The architecture consists of three primary components: a Warewulf host server that provisions operating system images, control plane nodes that manage the Kubernetes cluster, and worker nodes that execute containerised workloads.

Unlike traditional Kubernetes deployments, where nodes maintain persistent state, Ephemeral Kubernetes nodes boot from network-provisioned images and lose all local state upon reboot. The system leverages two distinct Warewulf images:

- A **control image** containing Kubernetes control plane components [6] (`kube-apiserver`, `kube-scheduler`, `kube-controller-manager`, `etcd`), HAProxy, and Keepalived for high availability.
- A **base image** for worker nodes containing only the `kubelet` and container runtime.

Both images include pre-loaded container images to enable offline deployment without internet connectivity.

A critical architectural component is the shared Network File System (NFS) storage mounted at `/share` on all nodes. This shared storage serves multiple purposes: synchronising configuration between nodes, storing pre-loaded container images, coordinating leader election during cluster bootstrap, and distributing cryptographic credentials required for cluster formation.

2.2 Cluster Formation and Node Join Process

The cluster formation process in Ephemeral Kubernetes follows a coordinated sequence that relies heavily on the shared NFS storage for synchronisation.

2.2.1 Leader Election & Cluster Initialization

When multiple control plane nodes boot simultaneously, a leader election mechanism determines which node initialises the cluster. The leader node creates a `/share/leader` folder on the NFS share. Other control nodes see that the `/share/leader` folder already exists, recognise themselves as followers and wait for cluster initialisation to complete.

The elected leader node performs several security-critical operations:

- **Certificate Generation:** The leader runs `kubeadm init`, which generates the cluster's Public Key Infrastructure (PKI) [7], including the Certificate Authority (CA) certificates, API server certificates, and etcd certificates.
- **Credential Creation:** Kubeadm generates the administrator configuration file (`kube.config`) [8] containing client certificates signed by the cluster CA. These certificates include the Common Name (CN) `kubernetes-admin` and Organisation (O) `system:masters`, granting unrestricted cluster access.
- **Credential Distribution:** The leader copies all generated certificates and the admin configuration file to the NFS share: `/share/pki/ca.crt`, `/share/pki/ca.key`, `/share/pki/etcd/ca.crt`, `/share/pki/etcd/ca.key`, and `/share/kube.config`.
- **Ready Signal:** After completing initialisation, the leader creates the `/share/leader_ready` file to signal other nodes.

2.2.2 Follower and Worker Node Join Process

Non-leader nodes poll for the `leader_ready` file every 5 seconds. Once detected, nodes retrieve credentials from NFS and join the cluster:

- **Control Plane Followers:** Copy CA certificates from `/share/pki/` to `/etc/kubernetes/pki/`, start the Phylactery service for HAProxy configuration management, and use the admin configuration to join as control plane members using `kubeadm join -discovery-file /root/.kube/config -control-plane -v=5`.
- **Worker Nodes:** Read the admin configuration from `/share/kube.config` and join the cluster using `kubeadm join -discovery-file /share/kube.config`.

Both node types use the same administrator credentials for cluster joining.

2.3 Node Rejoin and Recovery Mechanism

A distinguishing feature of Ephemeral Kubernetes is its handling of node reboots. Since nodes are stateless, every time a node goes down, it wipes its local state, so a rebooted node appears as a new entity attempting to join an existing cluster. However, the cluster's internal state retains records of the node from its previous session, causing hostname and IP conflicts that prevent rejoining. The Phylactery service addresses this by performing cleanup operations to remove these stale memberships before the node attempts to join again. This recovery mechanism explicitly requires the `'kube.config'` file to be persistently available on the NFS share. Without the administrator-level credentials provided by `'kube.config'`, the service cannot authenticate with the API to purge the previous entries from the etcd database and the Kubernetes node list.

3 Security Vulnerabilities in the Ephemeral Model

As mentioned in the Node Rejoin section, all the nodes in the Kubernetes cluster plane need to have persistent access to cluster config files and certificates, which exist on `/share` folder on the cluster manager, which can give admin-level access to the Kubernetes cluster to any user on any node.

- **Privilege Escalation:** Stolen credentials enable deployment of privileged containers [9], leading to root access on any cluster node. A user with admin access to a Kubernetes cluster can escalate this to root access on any node of the cluster, for example, by deploying a privileged container that mounts `/etc/shadow` and replaces the root password.
- **No Revocation Mechanism:** Once credentials are copied, they cannot be revoked or expired. If an attacker copies the `kube.config` file, there is no way to invalidate those credentials without recreating the entire cluster.
- **Lateral Movement:** Compromising a single worker node provides a path to compromise the entire cluster. The shared credential model means that breaching any single node's security gives an attacker the keys to the entire infrastructure.

4 PKI Certificates and Their Usage

Kubernetes uses certificates and keys to secure the cluster and verify identities. Each certificate serves a specific purpose in the cluster's security model.

- **Cluster CA (`ca.crt/ca.key`):** The master certificate for the entire cluster. The `ca.crt` file lets nodes verify they're talking to the real API server. The `ca.key` file is used to create certificates for all cluster components. This is the most critical key - anyone with `ca.key` can create admin credentials and take over the cluster.
- **Service Account Keys (`sa.key/sa.pub`):** These keys create and verify tokens for pods running in the cluster. When a pod needs to talk to the API server, it gets a token signed with `sa.key`. The API server checks this token using `sa.pub`; all control plane nodes must use identical keys so tokens work no matter which API server receives the request.
- **Front Proxy CA (`front-proxy-ca.crt/front-proxy-ca.key`):** Used for API aggregation, which lets additional API servers connect to the main Kubernetes API. All control plane nodes must share these certificates for extension APIs to work correctly.
- **etcd CA (`etcd/ca.crt/etcd/ca.key`):** Protects communication between etcd database instances. When a new control plane node joins, it uses `etcd/ca.key` to create certificates that let it communicate securely with existing members.

- **Admin Kubeconfig File:** The `kube.config` file contains the cluster CA certificate, a client certificate proving admin identity, and a private key. Possession of this file grants complete administrative control over the cluster.

The current setup stores `ca.key` and `Kube.config` in the shared `/share` directory, which creates a security risk. Anyone accessing this folder can use `ca.key` or `Kube.config` to generate admin certificates and gain full cluster access. A better approach would use secured folder for storing these sensitive files, which is accessible only with some authentication, or share only limited-permission join tokens and service account tokens that only allow specific operations like node management.

5 Proposed Security Enhancement: Key-Based Credential Distribution

To address the critical security vulnerabilities inherent in the NFS-based credential distribution model, we propose a Key-based authentication system that eliminates persistent credential exposure while maintaining the operational benefits of ephemeral cluster deployment.

5.1 Architecture Overview

The enhanced architecture replaces the shared NFS storage of sensitive credentials with a secure file server running on the Warewulf cluster manager. This server implements a Key-based authentication mechanism where each node receives a unique Key that grants access to the credentials and config file.

The secure file server operates as a Flask-based HTTP service listening on the cluster manager's internal network interface (typically `10.0.0.3:8000`). Unlike the previous NFS-based approach, where credentials remained indefinitely accessible at `/share/pki/` and

`/share/kube.config`, the new system stores all sensitive materials in a private directory (`/var/lib/warewulf/private-secrets`) with strict filesystem permissions (mode 700), preventing access by any user or process except root, and the server running on the cluster manager will serve those files to anyone with a valid key.

5.2 Key Generation and Distribution

Prior to cluster deployment, the cluster administrator executes a Key generation script (`generate-Keys.sh`) that creates unique SHA-256 Keys for each node registered in Warewulf. Each Key is stored as a JSON file in `/var/lib/warewulf/Keys/` containing metadata including the associated node name, creation timestamp, and expiration status.

The Keys are distributed to nodes through Warewulf's overlay system. During node provisioning, each node receives its unique Key via a node-specific file (`/etc/k8s-Key.{NODE_NAME}`) injected into the boot image. This approach ensures Keys are delivered securely during the provisioning process without network transmission or shared storage exposure.

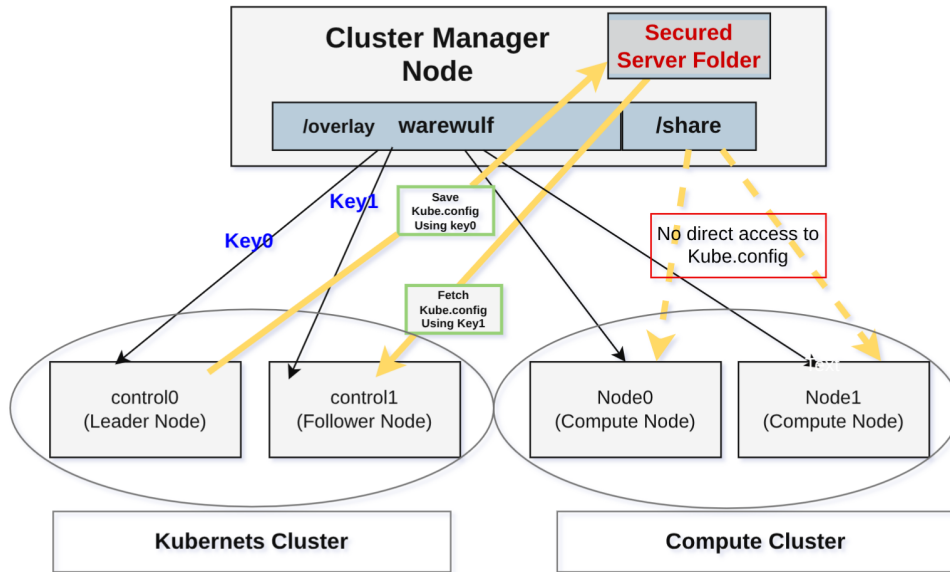


Figure 2: Kube.config will be saved in a secure server folder on the cluster manager, which is accessible using Key(separate for each node, provided through overlay) authentication. Figure created using <https://app.diagrams.net/>

5.3 Secure Credential Upload Mechanism

The leader node, after completing `kubeadm init` and generating the cluster PKI, uploads sensitive materials to the secure file server rather than copying them to NFS storage. The upload process uses HTTP POST requests with binary payloads:

```
curl -X POST --data-binary @/etc/kubernetes/kube.config \
    "http://10.0.0.3:8000/upload/kube.config/$Key"
```

The server validates the Key before accepting the upload and sanitizes filenames to prevent directory traversal attacks. Only two files are uploaded: the cluster administrator configuration (`kube.config`) and a compressed archive of CA certificates (`pki.tar.gz`) containing exclusively:

- Cluster CA certificate and Key (`ca.crt`, `ca.Key`)
- Service account Keys (`sa.Key`, `sa.pub`)
- Front proxy CA certificate and Key (`front-proxy-ca.crt`, `front-proxy-ca.Key`)
- etcd CA certificate and Key (`etcd/ca.crt`, `etcd/ca.Key`)

Critically, the archive excludes node-specific certificates (API server certificates, etcd peer certificates) that should be uniquely generated per node, preventing certificate reuse vulnerabilities.

5.4 Key-Authenticated Download and Self-Destruction

Follower nodes and worker nodes retrieve credentials by presenting their unique Keys to the secure file server:

```
curl -f -s "http://10.0.0.3:8000/config/$Key" \  
    -o /root/.kube/config  
curl -f -s "http://10.0.0.3:8000/certs/$Key" \  
    -o /tmp/pki.tar.gz
```

The server validates the Key's existence and ensures it has not been marked as expired. Upon successful credential download and cluster join completion, the node immediately deletes its Key from local storage, removing the file `/etc/k8s-Key.{NODE_NAME}`. This Key self-destruction ensures that once a node successfully joins the cluster, the authentication credential no longer persists in the node's memory or filesystem, eliminating the risk of Key theft from a running node.

5.5 Key Lifecycle and Node Rejoin Mechanism

The Key lifecycle is designed to accommodate the ephemeral nature of Warewulf-provisioned nodes. When a node reboots, it loses all state, including the previously used Key. However, since Warewulf re-provisions the node from the base image during each boot, the node receives the same Key through the overlay system. This Key, generated during the most recent execution of `generate-Keys.sh` allows the rebooted node to re-authenticate and download credentials again.

This process solves the problem of nodes "rejoining" a cluster after they reboot. Because the nodes are stateless and wipe their memory when they restart, every reboot is treated like a brand-new node joining the cluster for the first time. The node uses its Key to download credentials, joins the cluster (with the Phylactery service handling cleanup of the previous node instance), and subsequently deletes the Key. This cycle can repeat indefinitely across multiple reboots without accumulating credential exposure.

For operational deployments, administrators should regenerate Keys periodically (e.g., before planned maintenance windows or cluster redeployments) using the `generate-Keys.sh` script and rebuild the Warewulf overlays with `wwctl overlay build`. This ensures that Keys potentially compromised during previous cluster lifecycles cannot be reused.

5.6 Security Improvements

The Key-based system delivers several critical security enhancements compared to the NFS-based approach:

- **Elimination of Persistent Exposure:** Credentials are no longer stored indefinitely on a network-accessible filesystem. After the initial cluster formation, sensitive materials exist only in the private directory on the cluster manager, which is not exported via NFS and has filesystem-level access controls..
- **Transient Credential Presence:** Although the Key remains consistent across reboots until rotated, it is immediately deleted from the node's active filesystem after the join process completes. This ensures that a running node does not retain valid authentication credentials in memory or storage during its operation..
- **Node-Specific Authentication:** Each node authenticates using a unique Key tied to its identity. This enables audit logging of which nodes accessed credentials and when, facilitating forensic investigation of security incidents.

- **Revocation Capability:** Keys can be explicitly expired through the server's `/Key/expire` endpoint. Additionally, simply regenerating Keys and rebuilding overlays effectively revokes all previously issued Keys, providing a straightforward revocation mechanism.
- **Reduced Lateral Movement Risk:** Compromising a worker node no longer grants access to cluster-admin credentials. Even if an attacker gains root access to a running node, the authentication Key has already been deleted post-join, preventing credential theft.
- **Time-Bounded Attack Window:** The attack window for Key theft is limited to the brief period between node boot and successful cluster join (typically under 90 seconds), after which the Key is destroyed. This drastically reduces the temporal attack surface compared to indefinite NFS exposure.

5.7 Implementation Considerations

The secure file server is deployed as a systemd service (`ww-secure-server.service`) that starts automatically with the cluster manager. The service logs all access attempts, including successful downloads and authentication failures, to `/var/log/ww-file-server.log`, providing an audit trail for security monitoring.

The installation script (`k8s-install.sh`) has been modified to detect the security mode via the `USE_SECURE_MODE` environment variable. When enabled, nodes use Key-authenticated downloads instead of direct NFS access. After successfully joining the cluster, the script removes the Key file to prevent persistence of authentication credentials on the node.

The Key deletion occurs immediately after the `kubeadm join` command completes successfully. This ensures that the credential has served its purpose and is no longer needed, eliminating any residual security risk from Key storage on the node filesystem. Since the node's filesystem is entirely RAM-based (tmpfs), a reboot would naturally clear the Key regardless, but explicit deletion provides defense-in-depth.

For rejoin scenarios where a node reboots and must rejoin an existing cluster, the Warewulf provisioning system automatically delivers the same authentication Key assigned to that node's overlay. This ensures the node can seamlessly rejoin without manual intervention. However, this means the Key remains valid until rotated. To enforce credential rotation, administrators must manually regenerate Keys and rebuild the overlays before nodes reboot..

6 Alternative Approach: Token-Based Access Control

Instead of sharing the `kube.config` file and sensitive certificate keys in the `/share` directory, a more secure approach uses limited-permission tokens and carefully selected certificates that follow the principle of least privilege.

6.1 Recommended Files for Shared Storage

To secure the cluster while maintaining the resilience required for HPC environments, this architecture shares only the specific credentials necessary for node lifecycle operations. Unlike the original configuration which exposed full administrative control, these files follow the principle of least privilege.

6.1.1 Bootstrap Join Token (`join-command.txt`)

This file contains the `kubeadm join` command equipped with a bootstrap token [10]. This token is required for all nodes (workers and control plane) to register with the cluster.

- **Scope:** The token's permissions are strictly limited to the node joining process.
- **Security:** It cannot be used to access cluster resources, read secrets, or perform administrative operations. Even if a malicious actor obtains this token, they cannot query the API server or modify the cluster state.

6.1.2 Certificate Key (`certificate-key.txt`)

This key is required exclusively for control plane follower nodes to join the cluster in a high-availability configuration. In the original architecture, sensitive private keys (`ca.key`, `etcd/ca.key`, `sa.key`, `front-proxy-ca.key`) and certificates (`ca.crt`, `etcd/ca.crt`, `etcd/server.crt`, `etcd/server.key`) were stored in plain text on NFS, directly accessible to any node or user with share access. The token-based approach eliminates this exposure by leveraging `kubeadm`'s built-in certificate encryption mechanism. During `kubeadm init`, the leader node encrypts all sensitive private keys into a secure bundle. The `certificate-key.txt` acts as the decryption password for this bundle. When a control plane follower joins or rejoins after a reboot, it presents this key to the API server, receives the encrypted bundle, and decrypts it locally to obtain the certificates it needs, ensuring raw private keys never reside on NFS at any point.

- **Scope:** Strictly limited to control plane nodes only. Worker nodes do not require this key as they never need to obtain or decrypt any certificate bundle.
- **Security:** Even if a malicious actor obtains this key, they must still actively interact with the API server to retrieve the encrypted bundle, generating auditable logs. Critically, raw private keys such as `ca.key` are never directly accessible on NFS, eliminating the primary attack vector of the original architecture.

6.1.3 Phylactery Service Account Token (`phylactery-token.txt`)

This token allows the Phylactery service to clean up stale node entries from the Kubernetes node list managed by the API server and from the `etcd` member database upon reboot and rejoin (unlike the original approach, where the service relied on the unrestricted `kube.config` to clear these entries). It carries minimal permissions restricted to node management (`list`, `delete`, `drain`), ensuring it cannot access cluster secrets or perform broader administrative tasks.

- **Scope:** Strictly limited to node lifecycle management operations only. The token can `list`, `drain`, and `delete` node objects from the Kubernetes node list but has no permissions beyond these specific operations.

- **Security:** Even if a malicious actor obtains this token, they cannot access cluster secrets, deploy or modify workloads, generate new credentials, or perform any administrative operations. The token provides no path to privilege escalation unlike the original `kube.config` which granted full cluster-admin access.

6.2 Security Benefits

This approach follows the principle of least privilege, where each shared credential grants only the minimum permissions required for its specific function. Unlike sharing `ca.key` or `kube.config`, none of these tokens can be used to generate new administrative credentials or gain full cluster access.

Key security improvements include:

- **Limited Permissions:** Bootstrap tokens allow only node joining; Phylactery tokens allow only node management. Neither provides access to workloads, secrets, or cluster.
- **No Privilege Escalation:** Compromising shared storage does not yield credentials capable of creating privileged containers, reading secrets, or generating new certificates.
- **Credential Segregation:** Administrative credentials remain isolated on the leader node, preventing the primary attack vector identified in the vulnerability analysis.

This token-based access control model can be combined with the secure token delivery system from Section 5, where even these limited credentials are distributed through the authenticated file server rather than persistent NFS storage, providing defense-in-depth protection.

7 Discussion

This study analyzed the security posture of Ephemeral Kubernetes in HPC environments, identifying a critical vulnerability in its reliance on shared NFS storage for cluster bootstrapping. While the original architecture prioritises deployment speed and operational simplicity, achieving sub-90-second boot times, it does so by violating fundamental security principles. The centralised exposure of the `kube.config` file and the Cluster CA key (`ca.key`) creates a single point of failure where the compromise of any worker node or the shared file system results in the total compromise of the Kubernetes control plane. This security model is unsuitable for multi-tenant HPC centres where workload isolation is paramount.

To mitigate these risks, we proposed two distinct architectural improvements. The first approach, **Key-Based Credential Distribution**, addresses the *transport and storage* vector. By replacing the passive NFS share with an active, authenticated file server, we eliminate the persistent availability of credentials. The use of unique, node-specific keys distributed via Warewulf overlays ensures that authentication is tied to hardware identity, while the "self-destruct" mechanism (deleting keys post-join) significantly narrows the temporal attack window. This approach transforms the credential distribution from a static, globally accessible resource into a transient, audited transaction.

The second approach, **Token-Based Access Control**, addresses the *privilege scope* vector. By adhering to the principle of least privilege, this method replaces the omnipotent administrator credentials with scoped Bootstrap Tokens and Service Account tokens. Even if these tokens are intercepted, an attacker’s capabilities are strictly limited to node registration or specific cleanup tasks, preventing them from accessing secrets, escalating privileges, or dominating the cluster.

While both approaches independently improve security, they operate at different layers of the infrastructure. The Key-based system secures the *delivery pipeline*, protecting the credentials during transit and storage. The Token-based system secures the *payload*, ensuring that the credentials themselves grant minimal access. For production HPC environments requiring defence-in-depth, the optimal configuration would likely combine these methodologies: using the secure Key-based server to deliver the restricted Tokens. This combination would ensure that credentials are not only difficult to intercept but also hold limited value if compromised, thereby enabling Ephemeral Kubernetes to meet the rigorous security standards of modern supercomputing facilities without sacrificing its dynamic capabilities.

8 Conclusion

Ephemeral Kubernetes offers a promising solution for integrating dynamic container orchestration into traditional HPC environments, though its original design [1] left significant security challenges unaddressed. As noted in the foundational work, the reliance on unsecured NFS storage for credential distribution creates a single point of failure where the compromise of any node can lead to total cluster takeover. This report has presented concrete architectural solutions to these previously identified vulnerabilities. We introduced a Key-based delivery system to secure the transport and storage of credentials, alongside a Token-based access control model that minimizes the privileges granted to joining nodes. By implementing these proposed enhancements, HPC centers can effectively close the security gaps inherent in the original Ephemeral Kubernetes model, enabling safe, multi-tenant operation while retaining the rapid provisioning capabilities that make the platform unique.

9 References

1. Decker, J., Kunkel, J. Ephemeral Kubernetes: dynamically deleting and recreating clusters using Warewulf. J Supercomput 81, 1491 (2025). <https://doi.org/10.1007/s11227-025-07668-y>
2. Chazapis, A., Nikolaidis, F., Marazakis, M., Bilas, A. (2023). Running Kubernetes Workloads on HPC. In: High Performance Computing. ISC 2023. https://doi.org/10.1007/978-3-031-40843-4_14
3. Decker J, Metje S, Kunkel J (2025) Running Kubernetes workloads on rootless HPC systems using Slurm. https://www.thinkmind.org/library/CLOUD_COMPUTING/CLOUD_COMPUTING_2025/cloud_computing_2025_2_60_20036.html

4. Feng Liu, Kate Keahey, Pierre Riteau, and Jon Weissman. (2019). Dynamically negotiating capacity between on-demand and batch clusters. SC '18. <https://doi.org/10.1109/SC.2018.00041>
5. Zhou, N., Georgiou, Y., Pospieszny, M. et al. Container orchestration on HPC systems through Kubernetes. J Cloud Comp 10, 16 (2021). <https://doi.org/10.1186/s13677-021-00231-z>
6. Kubernetes Documentation. Kubernetes Components. <https://kubernetes.io/docs/concepts/overview/components/>
7. Kubernetes Documentation. PKI certificates and requirements. <https://kubernetes.io/docs/setup/best-practices/certificates/>
8. Kubernetes Documentation. Organizing Cluster Access Using kubeconfig Files. <https://kubernetes.io/docs/concepts/configuration/>
9. Kubernetes Documentation. Pod Security Standards. <https://kubernetes.io/docs/concepts/security/pod-security-standards/>
10. Kubernetes Documentation. Authenticating with Bootstrap Tokens. <https://kubernetes.io/docs/reference/access-authn-authz/bootstrap-tokens/>

A Work sharing

If you worked in a group, describe here how you distributed the work and the actual contributions of each peer.

A.1 Hans

...

A.2 Peter

...

B Code samples

This is part of the appendix...